

# CFSS GLOBAL INTERNSHIP 2026

## SOC Operations & Threat Detection Mastery

### Week 1 Report — Lab Architecture & Deployment

Intern: Israel Mbiyavanga David

|          |                                            |
|----------|--------------------------------------------|
| Domain   | SOC Analyst & Blue Teaming                 |
| Company  | CFSS (Cyber Security & Forensics Services) |
| Duration | 4 Weeks — Week 1 Submission                |
| Deadline | June 26, 2026 (11:59 PM)                   |
| Wazuh    | v4.14.5                                    |

## 1. Executive Summary

During Week 1 of the CFSS SOC Operations & Threat Detection Mastery internship, **Israel Mbiyavanga David** built a fully functional Security Operations Center (SOC) lab environment from scratch using open-source tools on two virtual machines. An Ubuntu 22.04 LTS server was deployed as the Wazuh SIEM stack (Manager + Indexer + Dashboard), and a Windows 10 Home endpoint was enrolled as a monitored agent. By the end of the week the agent was confirmed **Active** on the Wazuh Dashboard — fulfilling the Week 1 deliverable.

## 2. Lab Environment

| Component          | Details                                                |
|--------------------|--------------------------------------------------------|
| Hypervisor         | Oracle VirtualBox                                      |
| VM 1 — SIEM Server | Ubuntu 22.04 LTS (Wazuh Manager + Indexer + Dashboard) |
| VM 2 — Endpoint    | Windows 10 Home Single Language                        |
| Network            | Host-Only Adapter — 192.168.134.x subnet               |
| Server IP          | 192.168.134.133                                        |
| Wazuh Version      | v4.14.5                                                |

## 3. Methodology

### Step 1 — Install VirtualBox & Create Virtual Machines

Oracle VirtualBox was installed on the host machine. Two VMs were created and configured with a Host-Only Network Adapter so they could communicate on the 192.168.134.x subnet:

- **VM1 (Ubuntu 22.04):** 4 GB RAM · 2 vCPU · 50 GB storage — hosts the Wazuh SIEM stack.
- **VM2 (Windows 10):** 4 GB RAM · 2 vCPU · 60 GB storage — acts as the monitored endpoint.

### Step 2 — Deploy Wazuh Stack on Ubuntu VM

The Wazuh all-in-one installation assistant was used inside the Ubuntu VM terminal. After installation the server IP was verified:

```
curl -sO https://packages.wazuh.com/4.x/wazuh-install.sh sudo bash ./wazuh-install.sh -a ip a # Result: 192.168.134.133/24 on eth0
```

```
WAZUH Open Source Security Platform
https://wazuh.com
[wazuh-user@wazuh-server ~]$ sudo -i
[root@wazuh-server ~]# ip
Usage: ip [ OPTIONS ] OBJECT { COMMAND | help }
       ip [ -force ] -batch filename
where OBJECT := { address | addresslabel | fou | help | ila | ioam | l2tp | link |
                  macsec | maddress | monitor | mptcp | mroute | mrule |
                  neighbor | neighbour | netconf | netns | nexthop | ntable |
                  ntbl | route | rule | sr | stats | tap | tcpmetrics |
                  token | tunnel | tuntap | vrf | xfrm }
OPTIONS := { -V[ersion] | -s[tatistics] | -d[etails] | -r[esolve] |
             -h[uman-readable] | -iec | -j[son] | -p[retty] |
             -f[amily] { inet | inet6 | mpls | bridge | link } |
             -4 | -6 | -M | -B | -O |
             -l[oops] { maximum-addr-flush-attempts } | -echo | -br[ief] |
             -o[neline] | -t[imestamp] | -ts[hort] | -b[atch] [filename] |
             -rc[ubuf] [size] | -n[etns] name | -N[umeric] | -a[ll] |
             -c[olor]}
[root@wazuh-server ~]# ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
   link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
   inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
   inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
   link/ether 00:0c:29:c6:5c:6d brd ff:ff:ff:ff:ff:ff
   altname enp2s0
   altname ens32
   inet 192.168.134.133/24 metric 1024 brd 192.168.134.255 scope global dynamic eth0
       valid_lft 1223sec preferred_lft 1223sec
   inet6 fe80::20c:29ff:fe66:5c6d/64 scope link proto kernel_ll
       valid_lft forever preferred_lft forever
[root@wazuh-server ~]#
```

Screenshot 1 — Ubuntu terminal: server IP confirmed as 192.168.134.133 via ip a command

### Step 3 — Validate Services & Dashboard

After installation, all three Wazuh services were verified and the dashboard was accessed at <https://192.168.134.133>. Services were restarted in correct order to resolve initial startup delay:

```
sudo systemctl restart wazuh-indexer sudo systemctl restart wazuh-manager sudo systemctl restart wazuh-dashboard sudo systemctl status wazuh-dashboard # Active: running sudo systemctl status wazuh-indexer # Active: running
```

```
[root@uazuh-server ~]# systemctl status uazuh-dashboard
● uazuh-dashboard.service - uazuh-dashboard
   Loaded: loaded (/etc/systemd/system-uazuh-dashboard.service; disabled; preset: disabled)
   Active: active (running) since Wed 2026-06-03 08:15:57 UTC; 6min ago
     Main PID: 5720 (node)
       Tasks: 11 (limit: 4646)
      Memory: 266.0M
         CPU: 18.056s
     Group: /system.slice/uazuh-dashboard.service
    Drop-In: /usr/share/uazuh-dashboard/node/bin/node --no-warnings --max-http-header-size=65536 --unhandled-rejections=warn /usr/share/uazuh-dashboard/1
       CGroup: /system.slice/uazuh-dashboard.service
               └─5720 /usr/share/uazuh-dashboard/node/bin/node --no-warnings --max-http-header-size=65536 --unhandled-rejections=warn /usr/share/uazuh-dashboard/1

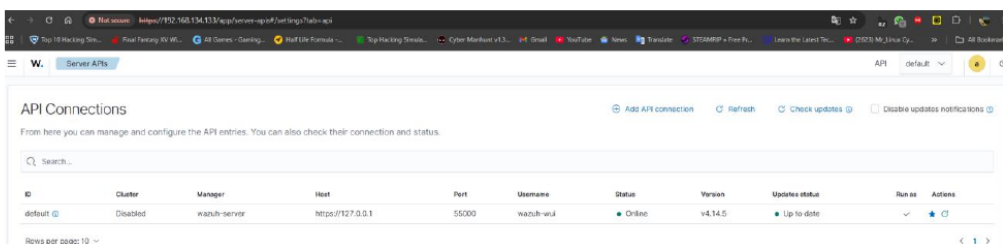
Jun 03 08:21:47 uazuh-server opensearch-dashboards[5720]: {"type":"log","@timestamp":"2026-06-03T08:21:47Z","tags":["error","opensearch","data"],"pid":"5720","m
Jun 03 08:21:49 uazuh-server opensearch-dashboards[5720]: {"type":"log","@timestamp":"2026-06-03T08:21:49Z","tags":["error","opensearch","data"],"pid":"5720","m
Jun 03 08:21:52 uazuh-server opensearch-dashboards[5720]: {"type":"log","@timestamp":"2026-06-03T08:21:52Z","tags":["error","opensearch","data"],"pid":"5720","m
Jun 03 08:21:54 uazuh-server opensearch-dashboards[5720]: {"type":"log","@timestamp":"2026-06-03T08:21:54Z","tags":["error","opensearch","data"],"pid":"5720","m
Jun 03 08:21:57 uazuh-server opensearch-dashboards[5720]: {"type":"log","@timestamp":"2026-06-03T08:21:57Z","tags":["error","opensearch","data"],"pid":"5720","m
Jun 03 08:21:59 uazuh-server opensearch-dashboards[5720]: {"type":"log","@timestamp":"2026-06-03T08:21:59Z","tags":["error","opensearch","data"],"pid":"5720","m
Jun 03 08:22:02 uazuh-server opensearch-dashboards[5720]: {"type":"log","@timestamp":"2026-06-03T08:22:02Z","tags":["error","opensearch","data"],"pid":"5720","m
Jun 03 08:22:04 uazuh-server opensearch-dashboards[5720]: {"type":"log","@timestamp":"2026-06-03T08:22:04Z","tags":["error","opensearch","data"],"pid":"5720","m
Jun 03 08:22:07 uazuh-server opensearch-dashboards[5720]: {"type":"log","@timestamp":"2026-06-03T08:22:07Z","tags":["error","opensearch","data"],"pid":"5720","m
Jun 03 08:22:09 uazuh-server opensearch-dashboards[5720]: {"type":"log","@timestamp":"2026-06-03T08:22:09Z","tags":["error","opensearch","data"],"pid":"5720","m
Jun 03 08:22:11 uazuh-server opensearch-dashboards[5720]: {"type":"log","@timestamp":"2026-06-03T08:22:11Z","tags":["error","opensearch","data"],"pid":"5720","m
[root@uazuh-server ~]# systemctl start uazuh-indexer
Synchronizing state of uazuh-indexer.service with SysV service script with /usr/lib/systemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable uazuh-indexer
[root@uazuh-server ~]# systemctl status uazuh-indexer

Jun 03 08:21:01 uazuh-server opensearch-dashboards[15720]: {"type":"log","@timestamp":"2026-06-03T08:21:01Z","tags":["error","opensearch","data"],"pid":"15720","m
Jun 03 08:21:04 uazuh-server opensearch-dashboards[15720]: {"type":"log","@timestamp":"2026-06-03T08:21:04Z","tags":["error","opensearch","data"],"pid":"15720","m

[root@uazuh-server ~]# systemctl status uazuh-dashboard
● uazuh-dashboard.service - uazuh-dashboard
   Loaded: loaded (/etc/systemd/system-uazuh-dashboard.service; disabled; preset: disabled)
   Active: active (running) since Wed 2026-06-03 08:15:57 UTC; 6min ago
     Main PID: 5720 (node)
       Tasks: 11 (limit: 4646)
      Memory: 266.0M
         CPU: 18.056s
     Group: /system.slice/uazuh-dashboard.service
    Drop-In: /usr/share/uazuh-dashboard/node/bin/node --no-warnings --max-http-header-size=65536 --unhandled-rejections=warn /usr/share/uazuh-dashboard/1
       CGroup: /system.slice/uazuh-dashboard.service
               └─5720 /usr/share/uazuh-dashboard/node/bin/node --no-warnings --max-http-header-size=65536 --unhandled-rejections=warn /usr/share/uazuh-dashboard/1

Jun 03 08:21:47 uazuh-server opensearch-dashboards[5720]: {"type":"log","@timestamp":"2026-06-03T08:21:47Z","tags":["error","opensearch","data"],"pid":"5720","m
Jun 03 08:21:49 uazuh-server opensearch-dashboards[5720]: {"type":"log","@timestamp":"2026-06-03T08:21:49Z","tags":["error","opensearch","data"],"pid":"5720","m
Jun 03 08:21:52 uazuh-server opensearch-dashboards[5720]: {"type":"log","@timestamp":"2026-06-03T08:21:52Z","tags":["error","opensearch","data"],"pid":"5720","m
Jun 03 08:21:54 uazuh-server opensearch-dashboards[5720]: {"type":"log","@timestamp":"2026-06-03T08:21:54Z","tags":["error","opensearch","data"],"pid":"5720","m
Jun 03 08:21:57 uazuh-server opensearch-dashboards[5720]: {"type":"log","@timestamp":"2026-06-03T08:21:57Z","tags":["error","opensearch","data"],"pid":"5720","m
Jun 03 08:21:59 uazuh-server opensearch-dashboards[5720]: {"type":"log","@timestamp":"2026-06-03T08:21:59Z","tags":["error","opensearch","data"],"pid":"5720","m
Jun 03 08:22:02 uazuh-server opensearch-dashboards[5720]: {"type":"log","@timestamp":"2026-06-03T08:22:02Z","tags":["error","opensearch","data"],"pid":"5720","m
Jun 03 08:22:04 uazuh-server opensearch-dashboards[5720]: {"type":"log","@timestamp":"2026-06-03T08:22:04Z","tags":["error","opensearch","data"],"pid":"5720","m
Jun 03 08:22:07 uazuh-server opensearch-dashboards[5720]: {"type":"log","@timestamp":"2026-06-03T08:22:07Z","tags":["error","opensearch","data"],"pid":"5720","m
Jun 03 08:22:09 uazuh-server opensearch-dashboards[5720]: {"type":"log","@timestamp":"2026-06-03T08:22:09Z","tags":["error","opensearch","data"],"pid":"5720","m
Jun 03 08:22:11 uazuh-server opensearch-dashboards[5720]: {"type":"log","@timestamp":"2026-06-03T08:22:11Z","tags":["error","opensearch","data"],"pid":"5720","m
[retr@uazuh-server ~]#
```

*Screenshot 2 — systemctl confirming wazuh-dashboard and wazuh-indexer services active and running*



Screenshot 3 — Wazuh API Connections: Status Online · Version v4.14.5 · Up to Date

## Step 4 — Enroll Wazuh Agent on Windows 10 VM

The agent deployment wizard was accessed via Wazuh Dashboard → Agents Management → Deploy new agent. Windows was selected as OS, Manager IP set to 192.168.134.133, and the generated PowerShell command was run on the Windows 10 VM:

```
Invoke-WebRequest -Uri https://packages.wazuh.com/4.x/windows/wazuh-agent-4.14.5-1.msi ^ -OutFile $env:tmp\wazuh-agent; msixexec.exe /i $env:tmp\wazuh-agent /q WAZUH_MANAGER='192.168.134.133' NET START WazuhSvc
```

✓

Server address:

This is the address the agent uses to communicate with the server. Enter an IP address or a fully qualified domain name (FQDN).

Assign a server address

192.168.134.133

☐ Remember server address

3

Optional settings:

By default, the deployment uses the hostname as the agent name. Optionally, you can use a different agent name in the field below.

Assign an agent name

Agent name

The agent name must be unique. It can't be changed once the agent has been enrolled.

Select one or more existing groups

Default

4

Run the following commands to download and install the agent:

Invoke-WebRequest -Uri https://packages.wazuh.com/4.x/windows/wazuh-agent-4.14.5-1.msi -OutFile \$env:temp/wazuh-agent.msi; cmd /c \$env:temp/wazuh-agent /q WAZUH\_MANAGER="192.168.134.133"

Wazuh

Groups

API default

Recently viewed

Home

Explore

groups, its agents and files.

Wazuh

Screenshot 4 — Wazuh Dashboard: agent deployment wizard with Manager IP 192.168.134.133

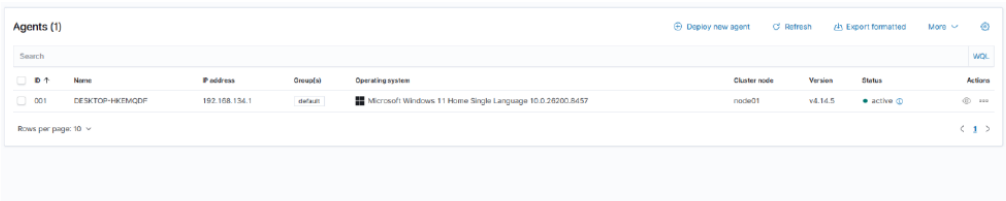
## 4. Technical Findings — Week 1 Deliverable

### 4.1 Active Agent Verification

After completing enrollment, the Wazuh Dashboard confirmed the Windows 10 endpoint as fully active and communicating with the Manager:

| Field            | Value                                                     |
|------------------|-----------------------------------------------------------|
| Agent ID         | 001                                                       |
| Agent Name       | DESKTOP-HKEMGDF                                           |
| IP Address       | 192.168.134.1                                             |
| Operating System | Microsoft Windows 10 Home Single Language 10.0.28200.8457 |
| Group            | default                                                   |
| Cluster Node     | node01                                                    |
| Version          | v4.14.5                                                   |
| Status           | ✓ Active                                                  |

### 4.2 Wazuh Dashboard — 1 Agent Active (Final Deliverable)



Screenshot 5 — FINAL DELIVERABLE: Wazuh Dashboard showing Agent 001 (DESKTOP-HKEMGDF) with Status: Active

### 4.3 Evidence Summary

| # | Screenshot       | What It Proves                                                |
|---|------------------|---------------------------------------------------------------|
| 1 | Server IP (ip a) | Ubuntu VM running at 192.168.134.133 — Wazuh server reachable |
| 2 | Services Status  | wazuh-dashboard and wazuh-indexer both Active and running     |
| 3 | API Connections  | Manager API Online, v4.14.5, Up to Date                       |

|   |                            |                                                          |
|---|----------------------------|----------------------------------------------------------|
| 4 | Agent Deploy Wizard        | Windows 10 agent correctly configured with Manager IP    |
| 5 | Dashboard — 1 Agent Active | Week 1 deliverable met: agent enrolled and communicating |

## 5. Key Learnings

---

- **Wazuh Architecture:** Manager (rule engine & event processing), Indexer (OpenSearch data store), and Dashboard (Kibana-based UI) work as three independent services.
- **Service Troubleshooting:** Read systemctl status outputs to diagnose startup issues; restart services in order: Indexer → Manager → Dashboard.
- **VirtualBox Networking:** Host-Only adapter creates a private subnet allowing VM-to-VM communication without exposing the lab to external networks.
- **Agent Enrollment:** The lifecycle is: Dashboard generates deploy command → MSI installed on Windows → agent registers to Manager on port 1514 → status becomes Active.
- **Real-World Simulation:** The two-VM setup mirrors actual enterprise SOC deployments with a central SIEM monitoring distributed endpoints.